Advanced Machine Learning, ETH Zurich

Attention

How a word reads the words around it

Carlos Cotrini

Autumn semester 2026

Entry 1

Words in context



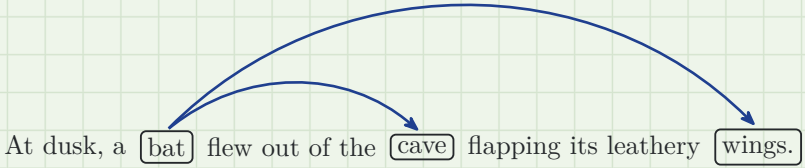
The question. In the language model, every row of E holds one word, yet the next word depends on the whole prefix. So row i must read the rows above it. How?

Now. Attention: every word reads the words around it and adds to its own row what it finds.

The same word can mean two different things. The words around it decide which one.

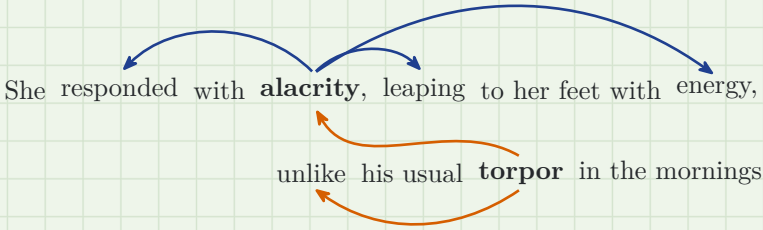


Here **bat** reads baseball and hit: a club.



Here **bat** reads cave and wings: an animal.

Few people know the words alacrity and torpor. Read the sentence: its other words tell you what they mean.



alacrity reads responded, leaping and energy: something quick and lively.

torpor reads unlike and alacrity: the opposite of alacrity.



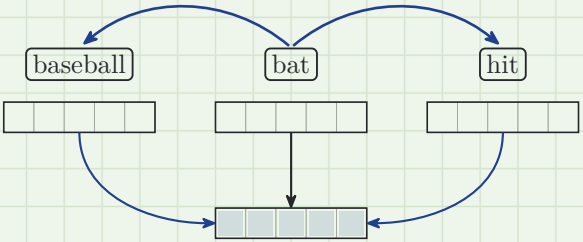
alacrity: brisk and cheerful readiness.



torpor: a state of physical or mental inactivity.

A network that reads a sentence does it in two steps, and every word does both.

1. Every word gets a representation: a row of numbers.



bat, now the club

2. Every word reads the other words and adds to its row what it finds.

This is *attention*. The rest of the lecture computes it by hand, on a language small enough to see every number.

You go shopping for a party. What you look for is the *query*; the signs over the shops are the *keys*; what you carry home are the *values*.

Query



Keys

GreenChoice Supermarket



StoryStack Books



Threadline Clothing Co



Candy Cave



Spirit Cellar Liquors



Values













The party matches three keys. Their values go home.

Entry 2

Wuggish



So far. A word takes its meaning from the words around it. Attention lets every word read the other words and add what it finds to its own representation.

Now. A toy language of eight words, Wuggish, and a way to write each word as numbers: a row of features, each a 1 or a 0.

Wuggish is a made-up sister language of English. A sentence describes up to three things, a cat, a snake and a chair, each plain or changed by its adjectives.



streept kat



bootet kat

The eight words

kat	cat	
snaak	snake	
chaar	chair	
streept	striped	cat or snake
bootet	booted	cat
vennomus	venomous	snake
wudden	wooden	chair
brooken	broken	chair



vennomus snaak wudden chaar

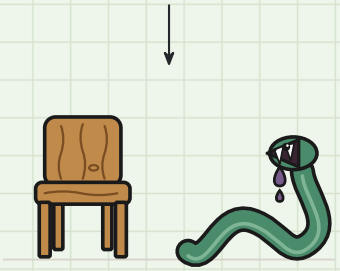


streept snaak streept kat wudden chaar

The order of the words carries nothing: *wudden chaar vennomus snaak* says the same. A sentence is a bag of words.

Learn the eight words from pictures, then read sentences whose words come in any order. At the end, the game shows what you have been doing all along.

wudden *snaak* *vennomus* *chaar*



Play it now, on a laptop or a phone: scan the code, or open the address below.

eth-aml-hs26.github.io/neural-networks/viz/wuggish-game

To compute with things, we ask every thing the same nine questions. Each question is a *feature*, and its value is 1 for yes and 0 for no.

	what kind of thing				how it looks				
	ANIMAL an animal?	PAWS paws?	REPTILE a reptile?	OBJECT an object?	STRIPES stripes?	BOOTS boots?	VENOM venom?	WOOD of wood?	BROKEN broken?
	**1**	**1**	0	0	0	**1**	0	0	0
	**1**	0	**1**	0	**1**	0	**1**	0	0
	0	0	0	**1**	0	0	0	**1**	**1**

Four features say what kind of thing it is, five say how it looks. Every number is a 0 or a 1, and the 1s tell the whole story.

Every Wuggish word gets one row of the nine features. A noun says what kind of thing it is, an adjective what it adds. The rows together form the *embedding table*.

	ANIMAL	PAWS	REPTILE	OBJECT	STRIPES	BOOTS	VENOM	WOOD	BROKEN
kat	**1**	**1**	0	0	0	0	0	0	0
snaak	**1**	0	**1**	0	0	0	0	0	0
chaar	0	0	0	**1**	0	0	0	0	0
streept	0	0	0	0	**1**	0	0	0	0
bootet	0	0	0	0	0	**1**	0	0	0
vennomus	0	0	0	0	0	0	**1**	0	0
wudden	0	0	0	0	0	0	0	**1**	0
brooken	0	0	0	0	0	0	0	0	**1**

8 words by $d = 9$ features

A word, a *token*, names a row. A feature names a column.

kat is an animal with paws, *snaak* an animal and a reptile. *bootet* adds boots, *streept* adds stripes.

We wrote these rows by hand, so that we can read them. A trained network learns its own rows, and its features have no names.

Add the rows of *kat* and *bootet*, column by column: out comes the booted cat. Every column stays a 0 or a 1.

	ANIMAL	PAWS	REPTILE	OBJECT	STRIPES	BOOTS	VENOM	WOOD	BROKEN
kat	1	1	0	0	0	0	0	0	0
+ *bootet*	0	0	0	0	0	1	0	0	0
	1	1	0	0	0	1	0	0	0



	ANIMAL	PAWS	REPTILE	OBJECT	STRIPES	BOOTS	VENOM	WOOD	BROKEN
chaar	0	0	0	1	0	0	0	0	0
wudden	0	0	0	0	0	0	0	1	0
+ *brooken*	0	0	0	0	0	0	0	0	1
	0	0	0	1	0	0	0	1	1



So a sentence is easy to read once we know which words go together. Which ones do?

Write the sentence one word per row and replace every word by its row of the table. The result is E, with $n = 7$ rows and $d = 9$ columns.

	ANIMAL	PAWS	REPTILE	OBJECT	STRIPES	BOOTS	VENOM	WOOD	BROKEN
wudden	0	0	0	0	0	0	0	**1**	0
kat	**1**	**1**	0	0	0	0	0	0	0
vennomus	0	0	0	0	0	0	**1**	0	0
chaar	0	0	0	**1**	0	0	0	0	0
bootet	0	0	0	0	0	**1**	0	0	0
snaak	**1**	0	**1**	0	0	0	0	0	0
brooken	0	0	0	0	0	0	0	0	**1**

$n = 7$ rows, $d = 9$ columns

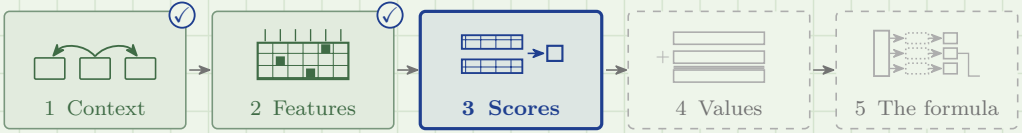
	ANIMAL	PAWS	REPTILE	OBJECT	STRIPES	BOOTS	VENOM	WOOD	BROKEN
kat	**1**	**1**	0	0	0	0	0	0	0
snaak	**1**	0	**1**	0	0	0	0	0	0
chaar	0	0	0	**1**	0	0	0	0	0
streept	0	0	0	0	**1**	0	0	0	0
bootet	0	0	0	0	0	**1**	0	0	0
vennomus	0	0	0	0	0	0	**1**	0	0
wudden	0	0	0	0	0	0	0	**1**	0
brooken	0	0	0	0	0	0	0	0	**1**

the embedding table

The row of *kat* says an animal with paws. Nothing in it says booted: *bootet* sits three rows below. Attention will make the row of *kat* read the row of *bootet*.

Entry 3

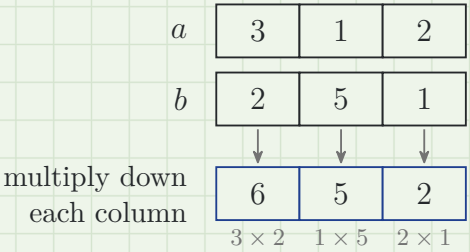
Queries, keys and scores



So far. Every word is a row of features, and the rows of a thing's words add up to the thing. In a sentence we do not know yet which words go together.

Now. Every word asks with a *query* and answers with a *key*. One number, the inner product of a query and a key, says whether two words go together.

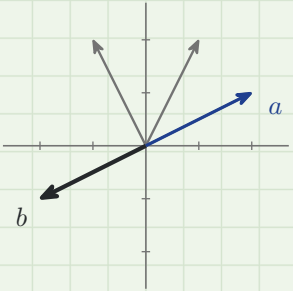
Two rows of the same length give one number, their *inner product*.



then add $6 + 5 + 2 = \boxed{13}$ the inner product of a and b

inner product of a and $b = a_1b_1 + a_2b_2 + a_3b_3$

Read a row of two numbers as an arrow. Keep a and turn b: the inner product falls as the angle between them opens.



$a = (2, 1)$

$b = (2, 1)$	$2 \times 2 + 1 \times 1 = 5$	same direction
$b = (1, 2)$	$2 \times 1 + 1 \times 2 = 4$	close directions
$b = (-1, 2)$	$2 \times (-1) + 1 \times 2 = 0$	at right angles
$b = (-2, -1)$	$2 \times (-2) + 1 \times (-1) = -5$	opposite directions

Every word gets a second row, its *query*: what it looks for in the other words. A query is written in the four features that say what kind of thing something is.

	ANIMAL	PAWS	REPTILE	OBJECT
wudden	0	0	0	0
kat	**1**	**1**	0	0
vennomus	0	0	0	0
chaar	0	0	0	**1**
bootet	0	0	0	0
snaak	**1**	0	**1**	0
brooken	0	0	0	0

Q

kat looks for the words that fit an animal with paws.

snaak looks for an animal and a reptile, *chaar* for an object.

An adjective is no thing, so it looks for nothing: its query is all zeros.

Q has $n = 7$ rows, one per word, and $k = 4$ columns, one per kind feature.

Every word gets a third row, its *key*: what it offers to the words that look. A key is written in the same four features as a query.

	ANIMAL	PAWS	REPTILE	OBJECT
wudden	0	0	0	**1**
kat	0	0	0	0
vennomus	0	0	**1**	0
chaar	0	0	0	0
bootet	0	**1**	0	0
snaak	0	0	0	0
brooken	0	0	0	**1**

K

bootet describes things with paws.

vennomus describes reptiles, *wudden* and *brooken* objects.

A noun describes no other word: its key is all zeros.

streept, missing from this sentence, describes any animal:

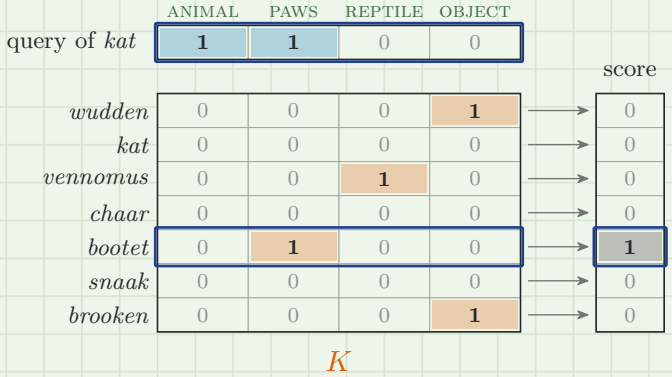
streept	**1**	0	0	0

Take the query of *kat* and the key of another word. Multiply down each column and add: every column where both rows hold a 1 adds 1.

	ANIMAL	PAWS	REPTILE	OBJECT	inner product	
query of *kat*	1	1	0	0		
key of *bootet*	0	1	0	0	1	they share PAWS
key of *streept*	1	0	0	0	1	they share ANIMAL
key of *vennomus*	0	0	1	0	0	they share nothing
key of *wudden*	0	0	0	1	0	

The inner product of two rows of zeros and ones counts the features both rows have.

Lay the query of *kat* over every key of the sentence and take the inner products. Each gives a *score*: how well that word fits *kat*.



bootet describes things with paws, and *kat* is one.

Every other word scores 0: their keys share no feature with the query of *kat*.

Stack the queries into Q and lay the keys on their side, $K^\top$. Their product holds every score at once: row i, column j scores the query of word i against the key of word j.

	ANIMAL	PAWS	REPTILE	OBJECT
wudden	0	0	0	0
kat	**1**	**1**	0	0
vennomus	0	0	0	0
chaar	0	0	0	**1**
bootet	0	0	0	0
snaak	**1**	0	**1**	0
brooken	0	0	0	0

Q, 7×4

	wudden	*kat*	*vennomus*	*chaar*	*bootet*	*snaak*	*brooken*
ANIMAL	0	0	0	0	0	0	0
PAWS	0	0	0	0	**1**	0	0
REPTILE	0	0	**1**	0	0	0	0
OBJECT	**1**	0	0	0	0	0	**1**

$K^\top$, 4×7

wudden	0	0	0	0	0	0	0
kat	0	0	0	0	**1**	0	0
vennomus	0	0	0	0	0	0	0
chaar	**1**	0	0	0	0	0	**1**
bootet	0	0	0	0	0	0	0
snaak	0	0	**1**	0	0	0	0
brooken	0	0	0	0	0	0	0

$S = QK^\top$, 7×7

Row *kat*, column *bootet*: the query of *kat* against the key of *bootet*. Score 1.

Four 1s: each adjective with the noun it describes. Every other score is 0.

Entry 4

Values



So far. The scores pair *kat* with *bootet*, *snaak* with *vennomus*, and *chaar* with *wudden* and *brooken*.

Now. Every word gives a *value*, what it adds to a thing, and every word adds to its own row the values of the words it scored 1 with.

Every word gets a fourth row, its *value*: what it adds to the word that reads it. A value is written in all nine features.

	ANIMAL	PAWS	REPTILE	OBJECT	STRIPES	BOOTS	VENOM	WOOD	BROKEN
wudden	0	0	0	0	0	0	0	**1**	0
kat	0	0	0	0	0	0	0	0	0
vennomus	0	0	0	0	0	0	**1**	0	0
chaar	0	0	0	0	0	0	0	0	0
bootet	0	0	0	0	0	**1**	0	0	0
snaak	0	0	0	0	0	0	0	0	0
brooken	0	0	0	0	0	0	0	0	**1**

V

bootet gives boots, *vennomus* venom, *wudden* wood and *brooken* broken.

A noun adds nothing to another thing: its value is all zeros.

V has $n = 7$ rows and $d = 9$ columns, one per feature.

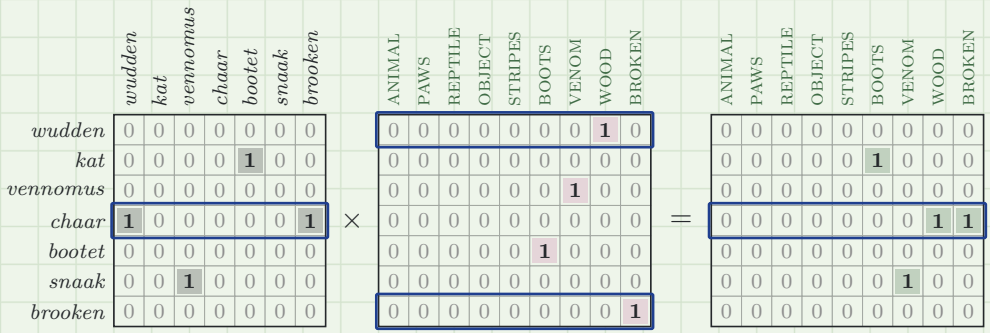
kat adds to its row the value of every word of the sentence, times the score of that word. Only *bootet* scored 1.

	ANIMAL	PAWS	REPTILE	OBJECT	STRIPES	BOOTS	VENOM	WOOD	BROKEN
row of *kat* in E	**1**	**1**	0	0	0	0	0	0	0
$+ 1 \times$ value of *bootet*	0	0	0	0	0	**1**	0	0	0
$+ 0 \times$ every other value	0	0	0	0	0	0	0	0	0
new row of *kat*	**1**	**1**	0	0	0	**1**	0	0	0



An animal with paws, wearing boots: the booted cat. Every number is still a 0 or a 1.

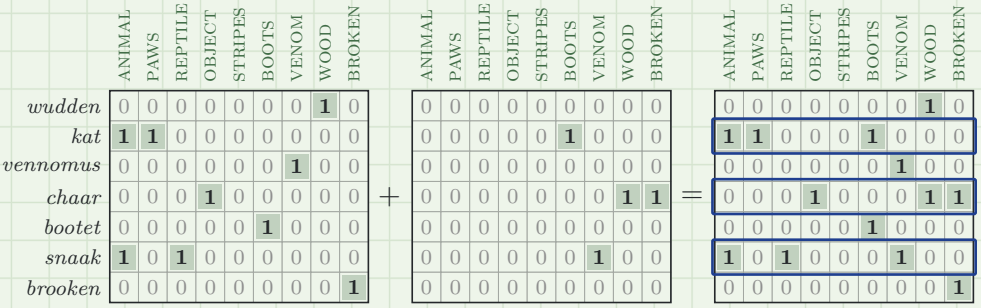
Row i of SV adds up the rows of V, each times its score in row i of S. One product does it for every word at once.



S, 7×7 V, 7×9 SV, 7×9

Row *chaar* of S: 1s at *wudden* and *brooken*. Row *chaar* of SV: wood and broken.

Add SV to E, row by row. A noun keeps its own features and gains the looks of its adjectives. An adjective found nothing, so it keeps its row as it was.



E SV $E + SV$, 7×9

The rows of the three nouns now carry their adjectives. Every number is still a 0 or a 1.

Read the new rows of *kat*, *chaar* and *snaak*. One layer of attention has read the sentence: every adjective has found its noun.

	ANIMAL	PAWS	REPTILE	OBJECT	STRIPES	BOOTS	VENOM	WOOD	BROKEN
wudden	0	0	0	0	0	0	0	**1**	0
kat	**1**	**1**	0	0	0	**1**	0	0	0
vennomus	0	0	0	0	0	0	**1**	0	0
chaar	0	0	0	**1**	0	0	0	**1**	**1**
bootet	0	0	0	0	0	**1**	0	0	0
snaak	**1**	0	**1**	0	0	0	**1**	0	0
brooken	0	0	0	0	0	0	0	0	**1**


a booted cat


a wooden, broken chair


a venomous snake

$E + SV$

One attention layer: n rows in, n rows out, each row enriched by the rows it scored 1 with.

In *kat streept snaak chaar wudden* one animal is striped. Which one? Compute the scores of the three nouns against the two adjectives.

	streept	*wudden*
kat	**1**	0
snaak	**1**	0
chaar	0	**1**

scores

streept answers ANIMAL, and both *kat* and *snaak* ask for it: 1 and 1, a tie.

	ANIMAL	PAWS	REPTILE	OBJECT	STRIPES	BOOTS	VENOM	WOOD	BROKEN
kat	**1**	**1**	0	0	**1**	0	0	0	0
snaak	**1**	0	**1**	0	**1**	0	0	0	0

new rows: both striped



what the scores give



what the sentence means: one of these

A bag of words keeps no order, and with it the only clue. A transformer adds to every row the position of its word, so that the scores can differ.

Entry 5

Attention in one formula



So far. We wrote every query, key and value by hand, and every new row came out as a 0 or a 1.

Now. A network computes Q, K and V from E with three blocks of weights, and turns every row of scores into weights that add up to 1.

A network does not get the queries for free: it computes them from E, with a block of weights W^Q. Here every weight is a 0 or a 1, so we can read them.

	ANIMAL	PAWS	REPTILE	OBJECT	STRIPES	BOOTS	VENOM	WOOD	BROKEN
wudden	0	0	0	0	0	0	0	**1**	0
kat	**1**	**1**	0	0	0	0	0	0	0
vennomus	0	0	0	0	0	0	**1**	0	0
chaar	0	0	0	**1**	0	0	0	0	0
bootet	0	0	0	0	0	**1**	0	0	0
snaak	**1**	0	**1**	0	0	0	0	0	0
brooken	0	0	0	0	0	0	0	0	**1**

E, 7×9

$\times$

	ANIMAL	PAWS	REPTILE	OBJECT
ANIMAL	**1**	0	0	0
PAWS	0	**1**	0	0
REPTILE	0	0	**1**	0
OBJECT	0	0	0	**1**
STRIPES	0	0	0	0
BOOTS	0	0	0	0
VENOM	0	0	0	0
WOOD	0	0	0	0
BROKEN	0	0	0	0

W^Q, 9×4

$=$

	ANIMAL	PAWS	REPTILE	OBJECT
	0	0	0	0
	1	**1**	0	0
	0	0	0	0
	0	0	0	**1**
	0	0	0	0
	1	0	**1**	0
	0	0	0	0

Q, 7×4

Each column of W^Q holds one 1: it copies one kind feature. The row of *kat* keeps ANIMAL and PAWS; an adjective has no kind feature, so its query is all zeros.

The keys come from E the same way, with a second block of weights, W^K. Its rows say which kind of thing each look goes with.

	ANIMAL	PAWS	REPTILE	OBJECT	STRIPES	BOOTS	VENOM	WOOD	BROKEN
wudden	0	0	0	0	0	0	0	**1**	0
kat	**1**	**1**	0	0	0	0	0	0	0
vennomus	0	0	0	0	0	0	**1**	0	0
chaar	0	0	0	**1**	0	0	0	0	0
bootet	0	0	0	0	0	**1**	0	0	0
snaak	**1**	0	**1**	0	0	0	0	0	0
brooken	0	0	0	0	0	0	0	0	**1**

E, 7×9

$\times$

	ANIMAL	PAWS	REPTILE	OBJECT
ANIMAL	0	0	0	0
PAWS	0	0	0	0
REPTILE	0	0	0	0
OBJECT	0	0	0	0
STRIPES	**1**	0	0	0
BOOTS	0	**1**	0	0
VENOM	0	0	**1**	0
WOOD	0	0	0	**1**
BROKEN	0	0	0	**1**

W^K, 9×4

$=$

ANIMAL	PAWS	REPTILE	OBJECT
0	0	0	**1**
0	0	0	0
0	0	**1**	0
0	0	0	0
0	**1**	0	0
0	0	0	0
0	0	0	**1**

K, 7×4

Training must find such weights in the text.

Row STRIPES, column ANIMAL: stripes go on animals. Row by row, W^K holds the grammar of Wuggish.

A third block of weights, W^V, makes the values. It keeps the five features of a look and drops the four of a kind.

	ANIMAL	PAWS	REPTILE	OBJECT	STRIPES	BOOTS	VENOM	WOOD	BROKEN
wudden	0	0	0	0	0	0	0	**1**	0
kat	**1**	**1**	0	0	0	0	0	0	0
vennomus	0	0	0	0	0	0	**1**	0	0
chaar	0	0	0	**1**	0	0	0	0	0
bootet	0	0	0	0	0	**1**	0	0	0
snaak	**1**	0	**1**	0	0	0	0	0	0
brooken	0	0	0	0	0	0	0	0	**1**

E, 7×9

$\times$

	ANIMAL	PAWS	REPTILE	OBJECT	STRIPES	BOOTS	VENOM	WOOD	BROKEN
ANIMAL	0	0	0	0	0	0	0	0	0
PAWS	0	0	0	0	0	0	0	0	0
REPTILE	0	0	0	0	0	0	0	0	0
OBJECT	0	0	0	0	0	0	0	0	0
STRIPES	0	0	0	0	**1**	0	0	0	0
BOOTS	0	0	0	0	0	**1**	0	0	0
VENOM	0	0	0	0	0	0	**1**	0	0
WOOD	0	0	0	0	0	0	0	**1**	0
BROKEN	0	0	0	0	0	0	0	0	**1**

W^V, 9×9

$=$

	ANIMAL	PAWS	REPTILE	OBJECT	STRIPES	BOOTS	VENOM	WOOD	BROKEN
	0	0	0	0	0	0	0	**1**	0
	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	**1**	0	0
	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	**1**	0	0	0
	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	**1**

V, 7×9

The value of an adjective is its look; the value of a noun is all zeros. These are the values we used.

A real attention layer does one more thing: in every row it divides the scores by their total. Then the weights of a row add up to 1.

	wudden	*kat*	*vennomus*	*chaar*	*bootet*	*snaak*	*brooken*
kat	0	0	0	0	**1**	0	0
chaar	**1**	0	0	0	0	0	**1**

scores

÷ 1
÷ 2

	wudden	*kat*	*vennomus*	*chaar*	*bootet*	*snaak*	*brooken*
kat	0	0	0	0	**1**	0	0
chaar	$\frac{1}{2}$	0	0	0	0	0	$\frac{1}{2}$

weights

Each row adds up to 1.

	ANIMAL	PAWS	REPTILE	OBJECT	STRIPES	BOOTS	VENOM	WOOD	BROKEN
new row of *chaar*	0	0	0	**1**	0	0	0	$\frac{1}{2}$	$\frac{1}{2}$

kat still takes all of boots. *chaar* takes half of wood and half of broken: the same features, at half strength. We kept the raw scores so that every number stayed a 0 or a 1.

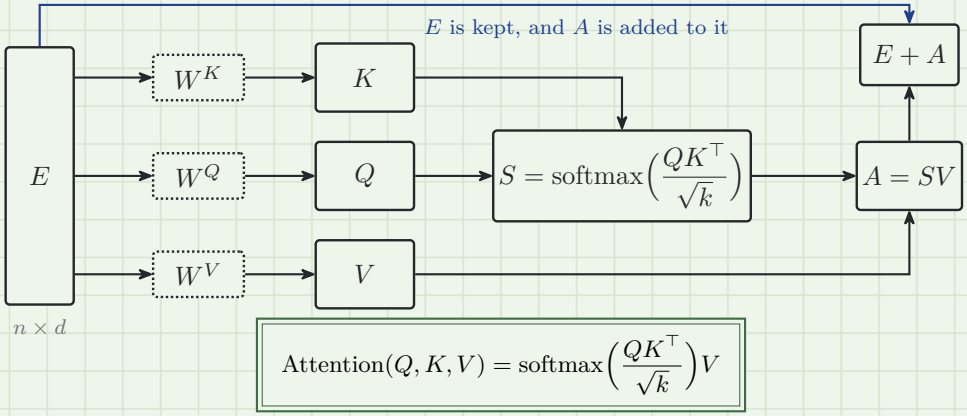
Instead of dividing the scores themselves, a transformer divides e^{score} by its total, the *softmax*. Every weight is then positive, and a large score takes almost everything.

$$\text{weight of word } j = \frac{e^{s_j}}{\sum_l e^{s_l}}$$

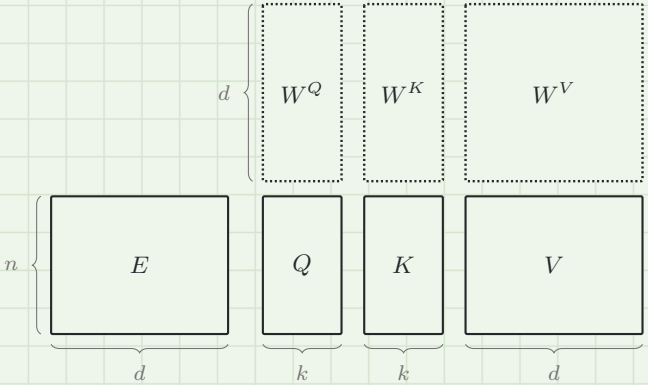
	wudden	*kat*	*vennomus*	*chaar*	*bootet*	*snaak*	*brooken*	
scores 0 and 1	0.13	0.13	0.13	0.13	**0.22**	0.13	0.13	With our scores of 0 and 1 the softmax barely chooses.
scores 0 and 9	0.01	0.01	0.01	0.01	**0.94**	0.01	0.01	With weights three times larger, a match scores 9 and takes almost all.

A trained network learns large scores for the words that belong together. Every score is first divided by $\sqrt{k}$, so that longer queries do not make it larger.

Three blocks of weights make Q, K and V from E. The softmax turns the scores into weights, the weights mix the values into A, and A is added to E.

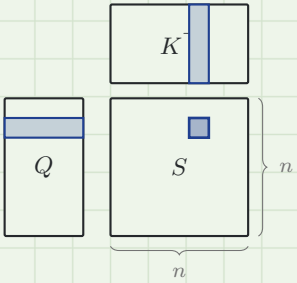


Every block drawn to scale, with this sentence's sizes: $n = 7$ words, $d = 9$ features, and $k = 4$ features in a query and a key.



every word against every word

S_{ij}: query i against key j



In a trained transformer nobody writes the weights: they are learned from text, and every head of every layer draws its own map of which words read which.

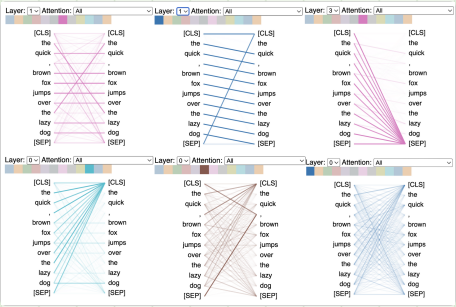


Image from https://www.comet.com/site/blog/explainable-ai-for-transformers/

One layer of attention, as we computed it on Wuggish.

1. Every word is a row of features: the sentence becomes E, $n \times d$.
2. A word asks with a query and answers with a key: $Q = EW^Q$ and $K = EW^K$.
3. The scores $S = QK^\top$: with rows of zeros and ones, a score counts the features a query and a key share.
4. The values $V = EW^V$: every word adds the values it scored, $E + SV$.
5. A transformer makes every row of scores add up to 1 with the softmax:

$$\text{Attention}(Q, K, V) = \text{softmax}\Big(\frac{QK^\top}{\sqrt{k}}\Big)V$$

Ties such as *streept*'s are broken by the position of every word, added to its row before Q, K and V are made.